

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Проектирование и реализация баз данных

Лабораторная работа № 3-4-1

Вариант 21

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлуниин Александр Александрович

г. Санкт-Петербург 2026 г

1. Цель работы

Использовать наработки из предыдущих этапов проектирования БД (статистика хоккейных матчей NHL) для создания прототипа поисковой и аналитической системы. Разработать программный скрипт на базе паттерна «Naïve RAG», который подключает большую языковую модель (LLM) к локальной базе данных MongoDB для предоставления точных ответов на вопросы пользователя естественным языком.

2. Используемые технологии

- **СУБД:** MongoDB Community Server (База данных: nhl_db, коллекция: games).
- **Языковая модель:** GigaChat API.
- **Язык программирования:** Python 3.
- **Ключевые библиотеки:** pymongo (драйвер БД), gigachat (SDK для работы с API), ruyaml (оптимизация сериализации), python-dotenv (безопасное управление конфигурацией).

3. Ход выполнения работы и реализация заданий

3.1. Базовая реализация (Подключение и извлечение данных)

Разработан скрипт, устанавливающий соединение с MongoDB. Программа запрашивает у пользователя уникальный идентификатор матча (`_id`, соответствующий `gamePk` в NHL API) и извлекает соответствующий документ. Организован интерактивный консольный интерфейс (цикл `while True`), позволяющий вести непрерывный диалог с ИИ-ассистентом по выбранному матчу.

3.2. Дополнительное задание №1 (Оптимизация формата

сериализации) Прямая передача JSON-документов в языковую модель приводит к перерасходу токенов контекстного окна из-за обилия служебных символов. Для решения этой проблемы перед отправкой данных реализована их конвертация в формат **YAML** с помощью библиотеки `ruyaml`. Иерархия на основе отступов делает данные более "легкими" для обработки нейросетью. Также программно удаляется техническое поле `_id`.

3.3. Дополнительное задание №2 (Препроцессинг данных и

Промпт-инжиниринг) Специфика спортивной статистики (наличие

массивов с множеством событий, авторами голов и ассистентами) вызывает у LLM проблемы с точностью вычислений (галлюцинации). Для решения этой архитектурной проблемы применен гибридный подход:

1. **Data Pre-processing (Препроцессинг на стороне Python):**

Реализована функция `build_goal_summary`. До передачи данных в LLM скрипт средствами Python обходит массив `goals`, вычисляет точное количество шайб для каждого игрока (игнорируя ассистентов) и распределяет голы по периодам. Эта готовая математическая выжимка внедряется в исходный документ под ключом `goal_summary`.

- ### 2. **Prompt Engineering:** Разработан строгий системный промпт, состоящий из 10 правил. Модели запрещено выдумывать факты, предписано строго доверять разделу `goal_summary` для любых статистических ответов, а также установлен жесткий запрет на перевод имен хоккеистов на русский язык (для сохранения оригинальных идентификаторов, например, "N. Kucherov"). Температура генерации снижена до 0.1.

3.4. Безопасность Ключ авторизации GigaChat API и строка подключения к БД вынесены в защищенный файл `.env`, что исключает компрометацию учетных данных при размещении кода в репозитории.

4. Листинг исходного кода (`main.py`)

```
import os
import yaml
from dotenv import load_dotenv
from pymongo import MongoClient
from gigachat import GigaChat
from gigachat.models import Chat, Messages, MessagesRole

load_dotenv()

GIGACHAT_CREDENTIALS = os.getenv("GIGACHAT_CREDENTIALS")
MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/")
DB_NAME = os.getenv("DB_NAME", "nhl_db")
COLLECTION_NAME = "games"

if not GIGACHAT_CREDENTIALS:
    raise ValueError("GIGACHAT_CREDENTIALS is not set")
```

SYSTEM_PROMPT = """Ты — профессиональный спортивный аналитик и ИИ-ассистент по статистике NHL.

Твоя задача: отвечать на вопросы пользователя, опираясь СТРОГО на предоставленные данные матча в формате YAML.

Правила:

1. Используй только ту информацию, которая есть в переданных данных. Не придумывай ничего от себя.
2. Если ответа на вопрос нет в предоставленном тексте, честно ответь: "В предоставленных данных нет информации об этом".
3. Формулируй ответ ясно, структурно и на грамотном русском языке.
4. Выступай в роли эксперта по хоккею.
5. Если вопрос связан с голами, авторами голов или количеством голов игроков, используй в первую очередь раздел goal_summary.
6. Авторами голов считаются только игроки из поля scorer_name.
7. Игроки из поля assists являются ассистентами, а не авторами голов.
8. Не добавляй ассистентов в список авторов голов.
9. Имена игроков выводятся строго так, как они указаны в данных: например, N. Kucherov, N. Paul, R. O'Reilly.
10. Не переводи имена игроков на русский язык и не раскрывай сокращённые имена."""

```
def connect_db():
```

```
    client = MongoClient(MONGO_URI)
```

```
    return client[DB_NAME][COLLECTION_NAME]
```

```
def build_goal_summary(doc: dict) -> dict:
```

```
    """
```

```
    Создаёт точную сводку по голам на основе поля goals.
```

```
    Это нужно, чтобы LLM не путала авторов голов и ассистентов.
```

```
    """
```

```
    goals = doc.get("goals", [])
```

```
    scorers_total = {}
```

```
    goals_by_period = {}
```

```
    for goal in goals:
```

```
        scorer_name = goal.get("scorer_name")
```

```
        period = goal.get("period")
```

```
        time = goal.get("time")
```

```
        team_id = goal.get("team_id")
```

```
        if not scorer_name:
```

```
            continue
```

```
        scorers_total[scorer_name] = scorers_total.get(scorer_name, 0) + 1
```

```

period_key = f"period_{period}"

if period_key not in goals_by_period:
    goals_by_period[period_key] = []

goals_by_period[period_key].append({
    "time": time,
    "team_id": team_id,
    "scorer_name": scorer_name
})

return {
    "important_note": "This section is calculated by Python. Use it for all questions about
goal scorers and number of goals. Do not count assists as goals.",
    "scorers_total": scorers_total,
    "goals_by_period": goals_by_period
}

def document_to_yaml(doc: dict) -> str:
    doc_copy = doc.copy()

    doc_copy["goal_summary"] = build_goal_summary(doc_copy)

    if '_id' in doc_copy:
        del doc_copy['_id']

    return yaml.dump(doc_copy, allow_unicode=True, default_flow_style=False,
sort_keys=False)

def ask_gigachat(yaml_data: str, user_question: str) -> str:
    user_content = f"Вот данные о хоккейном матче:\n{yaml_data}\n\nВопрос
пользователя: {user_question}"
    payload = Chat(
        messages=[
            Messages(role=MessagesRole.SYSTEM, content=SYSTEM_PROMPT),
            Messages(role=MessagesRole.USER, content=user_content)
        ],
        temperature=0.1,
        max_tokens=500,
    )
    with GigaChat(credentials=GIGACHAT_CREDENTIALS, verify_ssl_certs=False) as giga:
        response = giga.chat(payload)
        return response.choices[0].message.content

def main():

```

```

games_col = connect_db()
print("ИИ-Ассистент по статистике NHL (GigaChat)")
app_id_input = input("Введите ID матча для поиска (например, 2023020001): ").strip()
try:
    game_id = int(app_id_input)
except ValueError:
    print("Ошибка: ID матча должен быть числом.")
    return

game_doc = games_col.find_one({"_id": game_id})
if not game_doc:
    print(f"Матч с ID={game_id} не найден в базе данных.")
    return
print(f"\nМатч от {game_doc.get('date', 'Unknown')} найден!")
yaml_data = document_to_yaml(game_doc)
print("[Система] Данные успешно преобразованы в YAML.")
print("\n[Система] Чат запущен. Чтобы закончить, напишите 'выход', 'exit' или 'q'.")
while True:
    question = input("\nЧто вы хотите узнать об этом матче?\n> ").strip()
    if question.lower() in ['выход', 'exit', 'q', 'quit']:
        print("Завершение работы ИИ-Ассистента. До встречи!")
        break
    if not question:
        continue
    print("[Система] Отправка запроса в GigaChat...")
    answer = ask_gigachat(yaml_data, question)
    print("\nОтвет ИИ-Ассистента")
    print(answer)

if __name__ == "__main__":
    main()

```

5. Результаты выполнения скрипта

ИИ-Ассистент по статистике NHL (GigaChat)

Введите ID матча для поиска (например, 2023020001): 2023020001

Матч от 2023-10-10T21:30:00Z найден!

[Система] Данные успешно преобразованы в YAML.

[Система] Чат запущен. Чтобы закончить, напишите 'выход', 'exit' или 'q'.

Что вы хотите узнать об этом матче?

> Кто забивал голы в этом матче и сколько голов забил никита кучеров?

[Система] Отправка запроса в GigaChat...

Ответ ИИ-Ассистента

В представленном матче следующие игроки забивали голы:

- N. Kucherov (2 гола)
- R. O'Reilly (1 гол)
- J. Parssinen (1 гол)
- N. Paul (2 гола)
- B. Hagel (1 гол)
- T. Novak (1 гол)

Никита Кучеров забил 2 гола в этом матче.

Что вы хотите узнать об этом матче?

> выход

Завершение работы ИИ-Ассистента. До встречи!

Process finished with exit code 0

6. Вывод

В ходе выполнения лабораторной работы спроектирована и реализована система вопросно-ответного поиска (RAG) для сложной предметной области - статистики хоккейных матчей NHL. Выполнены оба задания на дополнительные баллы. Для обхода известной проблемы галлюцинаций LLM при подсчете элементов массивов был применен архитектурно верный паттерн: математическая агрегация и распределение данных выполнены на стороне бэкенда (Python), после чего языковая модель использовалась исключительно для генерации естественного текста по жестким правилам промпт-инжиниринга. Использование сериализации в YAML обеспечило экономию вычислительных ресурсов API.